

Hole in the Wall VR — Documentación del sistema

Guía para el equipo: qué hace cada parte, cómo se conecta todo y cómo levantarlo desde cero.

1. De qué se trata el proyecto

Réplica del juego de televisión *Hole in the Wall* (Human Tetris) aplicada a **rehabilitación motora**. Una pared con un agujero con forma de silueta avanza hacia el jugador, que tiene que adoptar esa postura con su cuerpo para pasar.

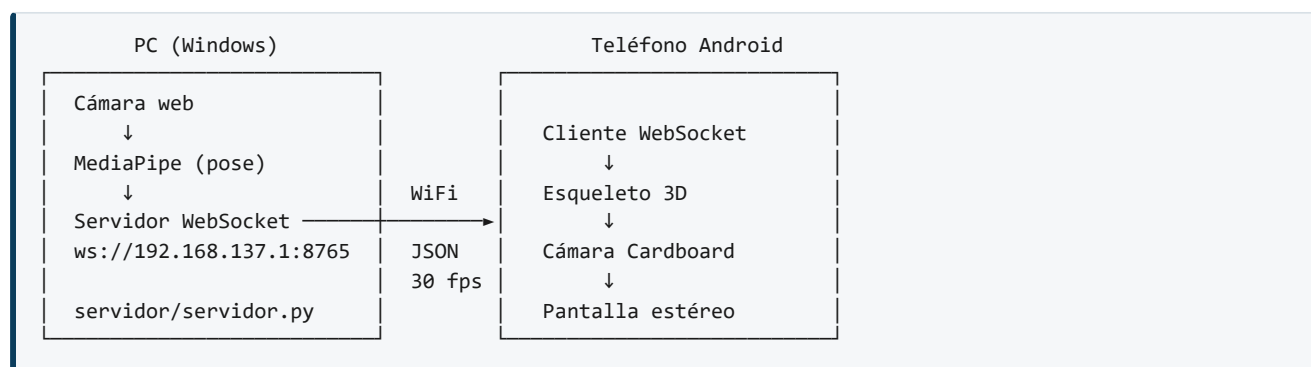
La diferencia con el juego original es el objetivo: acá las siluetas son **ejercicios de rehabilitación**. El paciente ve la pared en primera persona con un visor Cardboard, y una cámara común detecta su pose sin necesidad de sensores puestos encima. Eso trae dos consecuencias de diseño que conviene tener presentes al programar:

- **Las posturas tienen que ser alcanzables.** El rango de movimiento del paciente es limitado y variable. La tolerancia de acierto debe ser configurable, no una constante.
- **Nada de sobresaltos ni movimientos bruscos de cámara.** Un jugador con dificultad motora no puede corregir rápido, y el mareo en VR empeora todo.

Estado actual: el *framework* está terminado y probado en dispositivo (visor, tracking, calibración, red). **La lógica del juego todavía no existe** — no hay pared, ni agujero, ni evaluación de postura. Eso es lo que sigue.

2. Arquitectura general

Son dos programas separados que se hablan por WiFi:



La PC hace toda la visión por computadora (es la que tiene CPU), y el teléfono solo dibuja. El teléfono nunca ve la cámara: recibe **13 puntos por frame** y con eso arma el cuerpo.

El mensaje que viaja

JSON, ~30 veces por segundo:

```
{
  "landmarks": [
    { "id": 0, "name": "nose", "x": 0.505, "y": 0.727, "z": -0.952 },
    { "id": 11, "name": "left_shoulder", "x": 0.695, "y": 0.963, "z": -0.327 }
  ],
  "timestamp": 1785786061.45
}
```

- `x`, `y` → normalizados respecto del tamaño de la imagen.
- `y = 0` es **arriba** de la imagen (convención de imagen, no de 3D).
- `z` → profundidad relativa que estima MediaPipe.

⚠ `x` e `y` **PUEDEN SALIRSE del rango [0,1]**. Cuando el cuerpo se sale del encuadre, MediaPipe extrapola. Sentado y cerca de la cámara, los tobillos llegan a `y ≈ 2.4`. Si asumís [0,1] y centrás en 0.5, medio esqueleto termina metros bajo el piso. Es el error más fácil de cometer con estos datos.

Los 13 puntos usados: nariz (0), hombros (11,12), codos (13,14), muñecas (15,16), caderas (23,24), rodillas (25,26), tobillos (27,28).

3. Los archivos principales

3.1 Servidor — `servidor/servidor.py`

Único archivo del lado PC. Corre dos cosas en paralelo:

- **Hilo de MediaPipe** → abre la cámara, detecta la pose, guarda las posiciones en `pos_nodos`, y muestra una ventana de previsualización con los puntos dibujados encima (útil para verificar que te está viendo).
- **Loop asyncio** → cada 1/30 s toma `pos_nodos` y lo manda a todos los clientes conectados.

Detalle importante: usa la **API Tasks** de MediaPipe, no la vieja `mp.solutions`, que fue **eliminada en MediaPipe 1.0**. Por eso necesita el archivo de modelo `servidor/pose_landmarker_lite.task` (ya está en el repo). Si ves ejemplos en internet con `mp.solutions.pose`, son de una versión anterior y no funcionan.

3.2 Cliente y escena — `cardboardgodot/example/`

Archivo	Qué hace
<code>ExampleScene.tscn</code>	Escena principal. Contiene el piso, las torres y el Player.
<code>example_scene.gd</code>	El cliente WebSocket. Se conecta, recibe landmarks, arma los esqueletos, suaviza el movimiento y mueve la cabeza del jugador.
<code>Player.tscn</code>	El jugador: un <code>CharacterBody3D</code> con la cámara Cardboard adentro.
<code>player.gd</code>	Movimiento con teclado. <i>(Heredado del ejemplo del addon; hoy no se usa.)</i>

`example_scene.gd` es el archivo más importante para entender el sistema. Hace cuatro cosas:

1. **Descubrimiento del servidor.** Escanea la subred en la que está el propio teléfono, probando `.1`, `.2`, ... hasta `.20` en el puerto 8765. No tiene la IP hardcodeada a propósito: Android se cambia de red sola, así que fijar una IP se rompe seguido.
2. **Armado de los esqueletos.** Crea las esferas y los cilindros por código (`_armar_esqueleto`). Arma **dos**:
 - El **principal** (verde/celeste), que es el cuerpo real del jugador.
 - El de **debug** (naranja/rojo), 3 m adelante y a media escala, para poder mirar desde afuera qué movimiento está llegando. Se apaga con `ShowDebugSkeleton`.
3. **Suavizado (filtro One Euro).** Los datos de MediaPipe tiemblan. Un promedio simple obliga a elegir entre tembleque (poco filtro) o retardo (mucho), y en VR el retardo marea. El One Euro **varía el filtrado según la velocidad**: filtra fuerte cuando el punto está casi quieto —que es cuando el ruido se nota— y afloja al moverse rápido. Corre **una vez por frame**, no por paquete: los datos llegan a 30 Hz y la pantalla dibuja más rápido, así que aplicarlos al recibir dejaba escalones visibles.
4. **FollowHead**. El landmark 0 (la nariz) pasa a ser la cabeza del jugador: mover la cabeza frente a la cámara mueve el punto de vista en VR.

3.3 El addon de Cardboard — `cardboardgodot/addons/cardboard_vr/`

Archivo	Qué hace
<code>scripts/cardboard_vr_camera.gd</code>	El corazón del VR. Crea dos <code>SubViewport</code> (uno por ojo) con una cámara cada uno, aplica separación y convergencia, y lee el giroscopio.
<code>scenes/CardboardView.tscn</code>	La pantalla partida: dos <code>TextureRect</code> , uno por ojo, con el shader de lente.
<code>shaders/LensDistortion.gdshader</code>	Corrección de distorsión de la lente (modelo Brown-Conrady, coeficientes $k1/k2$).
<code>scripts/calibration_menu.gd</code>	Menú libre de calibración: permite tocar cualquier parámetro.
<code>scripts/calibration_wizard.gd</code>	Asistente guiado en dos fases para calibrar sin saber qué significa cada valor.

Cómo funciona el render estéreo: cada ojo se dibuja en su propio `SubViewport` desde una cámara desplazada lateralmente. Esas dos texturas se muestran lado a lado, y el shader las pre-deforma en barril para contrarrestar la curvatura de la lente física del visor. Sin esa corrección, las líneas rectas se ven curvas.

3.4 `cardboardgodot/cuerpo/`

Escenas viejas (`punto.tscn` , `segmento.tscn` , `munion.tscn`) de una versión anterior. **Hoy no se usan:** el cliente arma sus esqueletos por código. Están por si sirven de referencia.

4. Calibración del visor

Cada visor Cardboard y cada persona son distintos, así que hay dos herramientas adentro de la app. Se manejan con el **control VR bluetooth**.

Mapeo del control

El control expone sus 4 botones como **botones de joystick 0 a 3** (no como teclas, aunque otras apps los muestren como Enter/Back/etc.):

Control	Acción
Botón 1	Abrir / cerrar el menú de calibración
Botón 0	Resetear el parámetro seleccionado
Botones 2 / 3	Bajar / subir el valor
Botón 9	Recentrar la vista (volver al frente)
Botón 10	Abrir el asistente guiado
Palanca	Arriba/abajo elige parámetro, izq/der ajusta

El desplazamiento está rotado 270° por código, porque el control se agarra girado respecto de cómo reporta sus ejes.

Asistente guiado (botón 10)

- **Fase 1 — Distancia interpupilar.** Ajustar hasta ver una sola figura nítida, sin doble. Se ajustan separación y convergencia.
- **Fase 2 — Distorsión de lente.** Aparece una cuadrícula; ajustar k1 y k2 hasta que las líneas se vean rectas, sobre todo en los bordes.

Valores calibrados actuales

Sirven de punto de partida:

Parámetro	Valor
Separación de ojos	0.0
Altura de ojos	1.75 m
Ángulo de convergencia	8.9°
Sensibilidad giroscopio	0.024
Distorsión k1	0.205
Distorsión k2	0.5

Se guardan solos en el teléfono al cerrar el menú.

5. Cómo ejecutar todo

5.1 Requisitos

PC: - Python 3.13 - Godot **4.4.1** (para compilar el APK) - Android SDK con `adb` - Cámara web

Teléfono: Android con giroscopio, depuración USB activada, visor Cardboard y control bluetooth.

5.2 Instalar dependencias de Python

```
cd servidor
python -m pip install mediapipe websockets opencv-contrib-python
```

El modelo `pose_landmarker_lite.task` ya viene en el repo. Si faltara:

```
curl -L -o servidor/pose_landmarker_lite.task "https://storage.googleapis.com/mediapipe-models/pose_landmar
```

5.3 Crear la red WiFi en Windows 11

Este es el paso que más problemas da. La PC y el teléfono tienen que estar en la misma red; lo más simple es que la PC cree la suya.

1. **Configuración** → **Red e Internet** → **Zona con cobertura inalámbrica móvil** (*Mobile hotspot*).
2. Elegí **Wi-Fi** en "Compartir mi conexión a Internet desde".
3. **Editar** → poné nombre y contraseña.
4. Activá el interruptor.
5. En el teléfono, conectate a esa red.

Windows le asigna a la PC siempre la IP `192.168.137.1`, y al teléfono algo como `192.168.137.205`.

Para verificar desde la PC:

```
ipconfig
```

Tiene que aparecer `192.168.137.1` .

⚠ **Android se desconecta solo del hotspot.** Como esa red muchas veces no da internet, el teléfono se cambia solo a otra WiFi y deja de ver a la PC. Es la causa número uno de "dejó de andar". Si pasa, entrá a los ajustes de esa red en el teléfono y desactivá el cambio automático de red. Para comprobar en qué red está el teléfono: `bash adb shell ip addr show wlan0` Tiene que decir `192.168.137.x` .

⚠ **El Firewall de Windows puede bloquear el puerto 8765.** La primera vez que corras el servidor, Windows va a preguntar si permitís el acceso: **aceptá para redes privadas**. Si lo rechazaste sin querer, hay que habilitarlo a mano en Firewall de Windows Defender.

5.4 Levantar el servidor

```
cd servidor
python servidor.py
```

Tiene que abrirse la ventana **"MediaPipe Landmarks"** con la imagen de la cámara y los puntos verdes numerados encima. `Esc` la cierra.

La primera vez tarda unos segundos cargando el modelo y tira varios *warnings* de TensorFlow Lite (`Feedback manager requires a model with a single signature inference`) — son ruido normal, no errores.

5.5 Compilar e instalar el APK

Desde la carpeta del proyecto:

```
rm -rf cardboardgodot/android/build/build/
```

```
"C:/Program Files (x86)/Godot/Godot_v4.4.1-stable_win64.exe" --headless --path cardboardgodot --export-debu
```

```
adb install -r cardboardgodot/cardboardtest.apk
```

```
adb shell am start -n "com.example.cardboardgodot/com.godot.game.GodotApp"
```

Borrá siempre `android/build/build/` antes de exportar. El estado incremental viejo hace fallar el build con `processStandardDebugMainManifest` .

5.6 Verificar que conectó

```
adb logcat -s godot | findstr landmarks
```

Deberías ver:

```
[landmarks] Buscando servidor en 192.168.137.X:8765 ...  
[landmarks] Servidor: ws://192.168.137.1:8765  
[landmarks] Conectado a ws://192.168.137.1:8765  
[landmarks] Recibiendo (30 frames)
```

Y desde la PC, para confirmar que hay una conexión viva:

```
netstat -an | findstr 8765
```

6. Diagnóstico de problemas

Síntoma	Causa probable	Qué hacer
Sin servidor en ...	El teléfono se cambió de red	<code>adb shell ip addr show wlan0</code> ; tiene que ser <code>192.168.137.x</code>
Conecta y se corta	Firewall, o el hotspot se apagó	Revisar el Firewall; reactivar el hotspot
No se ve el esqueleto	La cámara no te detecta	Mirar la ventana de MediaPipe: ¿aparecen los puntos verdes?
El esqueleto aparece cortado o bajo el piso	y fuera de [0,1]	Alejarse de la cámara para entrar entero en el encuadre
La cámara no enciende (sin LED)	<code>VideoCapture(0)</code> tomó una cámara virtual	Si hay OBS o DroidCam instalados, cerrarlos
Todo tiembla	Filtro flojo	Bajar <code>MinCutoff</code> a 0.8 en el inspector
Se siente pastoso / con retardo	Filtro muy fuerte	Subir <code>MinCutoff</code> a 2.0 y <code>SmoothBeta</code> a 0.1
Se ve doble / no fusiona	Falta calibrar	Botón 10 → asistente guiado
La vista quedó torcida	Deriva del giroscopio	Botón 9 → recentrar

7. Lo que falta hacer

El framework está listo; **el juego no existe todavía**. Lo que viene:

1. **La pared con el agujero.** Una silueta 2D que avanza hacia el jugador.
2. **La evaluación de la postura.** Es la decisión de diseño más delicada: se puede comparar por *bounding box* (simple, tolerante, impreciso) o por solapamiento real de polígonos (exacto, mucho más exigente). **Para rehabilitación conviene empezar por lo tolerante** y agregar precisión después, con la tolerancia como parámetro configurable por paciente.

3. **Las siluetas de ejercicio**, definidas junto a quien sepa de rehabilitación.
4. **Realimentación al jugador**: qué parte del cuerpo está fuera del agujero.
5. **Registro de sesión**, para poder seguir el progreso del paciente.

Detalles conocidos, sin resolver

- `player.gd` referencia acciones de input que no existen (`jump` , `up` , `down` , `left` , `right`) y llena el log de errores cada frame. No rompe nada.
- Con `FollowHead` , la cabeza queda a ~0.55 m del piso en vez de los 1.75 configurados, porque la nariz llega en `y ≈ 0.73` y el esqueleto principal mapea centrado en 0.5. **Conviene resolverlo antes de programar la pared**, porque la lógica del juego depende de dónde está la cabeza.
- Las escenas de `cuerpo/` apuntan a rutas viejas. No se usan.

8. Referencia rápida de parámetros

Todos se editan en el inspector de Godot, en el nodo raíz de `ExampleScene` :

Parámetro	Default	Para qué
<code>ServerPort</code>	8765	Puerto del servidor
<code>SubnetBase</code>	<code>192.168.137.</code>	Subred del hotspot de Windows
<code>CloudDistance</code>	3.0	Distancia del esqueleto principal
<code>CloudScale</code>	2.0	Metros por unidad normalizada
<code>FollowHead</code>	true	La nariz controla el punto de vista
<code>MinCutoff</code>	1.2	Suavizado: más bajo = más suave
<code>SmoothBeta</code>	0.04	Suavizado: más alto = más reactivo
<code>HeadCutoffFactor</code>	0.6	Cuánto más se filtra la cabeza
<code>ShowDebugSkeleton</code>	true	Segundo esqueleto de debug
<code>DebugDistance</code>	3.0	Cuánto más adelante va el de debug